Fabula: AI-Generated Mystery Game — System Architecture and Technical Documentation

A Two-Phase Hybrid Architecture: Fine-Tuned LLM Generation, Classical-AI Simulation, and Live-API NPC Conversation

Semester AI Project Team

October 17, 2026

Abstract

Traditional video game Non-Player Characters (NPCs) suffer from either rigid, scripted predictability or unconstrained language-model hallucinations that violate game state and spatial reality. This report presents the architectural design and technical specification of **Fabula**, a small, single-player Python mystery game in which every NPC is AI-controlled and the entire game—mystery, town, cast, and each NPC's private knowledge and identity—is authored *before play* by a fine-tuned small (∼3–4B) language model. Generation runs locally in validated stages (world → cast → truth → per-NPC files) with grammar-constrained decoding and per-stage validators that retry or fail cleanly, producing a JSON game bundle. Play is a ∼10-minute session executed by a headless pure-Python engine: NPCs move via simplified schedules and A^* pathfinding (the "Body"), while free-text conversation is mediated exclusively by live APIs (the "Voice"), with each prompt assembled only from that NPC's own `identity.json` and `memory.json`. The engine keeps `truth.json` as ground truth that never enters any prompt or API call—a spoiler firewall enforced by tests. Consequently the fine-tuned model never runs at runtime, and evaluation targets *NPC behaviour* (secret-keeping, knowledge boundary, persona adherence, consistency) through scripted probe suites and a base-versus-fine-tune experiment.

Keywords: Fine-Tuned LLM, QLoRA, Knowledge Isolation, Spoiler Firewall, Live-API Dialogue, A^* Pathfinding, Mystery Game.

Contents

1 Project Overview **2**
- 1.1 Problem Formulation and Motivation . . . 2
- 1.2 Two-Phase Design . . . 2
- 1.3 Academic Relevance . . . 2

2 Inspiration and Related Work **3**
- 2.1 MuSR and the Synthetic-to-Natural Pipeline . . . 3
- 2.2 True Detective and Style References . . . 3
- 2.3 Superseded Design Lineage . . . 3

3 System Architecture and the Two-Phase Design **4**
- 3.1 Phase 1: Staged Generation (Local, Before Play) . . . 5
- 3.2 The Game Bundle . . . 5

3.3 Phase 2: The Play Loop (∼10 Minutes) . . . 5
3.3.1 Stage 1: World State (Ground Truth) . . . 6
3.3.2 Stage 2: Perception Filtering . . . 6
3.3.3 Stage 3: Internal NPC State . . . 6
3.3.4 Stage 4: Decision and Communication . . . 6
3.3.5 Stage 5: Action Execution and Feedback . . . 6

4 The Hybrid NPC Architecture: Classical AI and Live-API Integration 6
4.1 Search and Pathfinding (A^*) . . . 7
4.2 NPC Schedules (Body) . . . 7
4.3 Deliberately Excluded Components . . . 7
4.4 Runtime Conversation (Voice, Live APIs Only) . . . 7

5 NPC Knowledge, Secrets, and the Spoiler Firewall 9
5.1 Structured Facts, Not Free Text . . . 9
5.2 Knowledge Partitioning . . . 9
5.3 Authored Cover Stories . . . 10
5.4 The Spoiler Firewall . . . 10
5.5 Illustrative Example: Authored Information Asymmetry . . . 10

6 Data Generation and Fine-Tuning Pipeline 10
6.1 Fine-Tuning Configuration . . . 10
6.2 Baselines . . . 11

7 Implementation Plan and 8-Week Roadmap 11
7.1 Difficulty Assessment (Informal) . . . 11

8 Quantitative Evaluation Framework 12
8.1 Method: Scripted Probe Suite . . . 12
8.2 Core Evaluation Metrics . . . 12
8.3 Core Experiment . . . 13
8.4 Ablation Studies . . . 13
8.5 Judge and Human Sampling . . . 13

9 Summary and Key Advantages 13

1 Project Overview

1.1 Problem Formulation and Motivation

In interactive entertainment, Non-Player Characters have historically relied on Finite State Machines (FSMs) or static behavior trees. While computationally trivial and deterministic, these characters display zero episodic recall, fail to adapt to emergent events, and repeat canned conversational dialogue.

Recent advances in generative AI have prompted attempts to replace scripted logic entirely with Large Language Models (LLMs). However, pure LLM-driven agents introduce critical architectural flaws:

1. **Factual and Physical Hallucination:** Generative models invent unearned items, claim impossible physical actions, or bypass geographic and knowledge boundaries.
2. **Loss of Grounding:** Language models lack the game's state; without an external engine they cannot reliably track time, position, or who knows what.
3. **Excessive Computational Overhead:** Querying frontier models for every low-level decision is latency-heavy, costly, and fragile.

Fabula resolves this dichotomy with a **two-phase hybrid architecture**. Phase 1 (*Generation*) runs a fine-tuned small LLM locally, before the game starts, and produces a complete, validated JSON bundle: the mystery, a tiny town of **3–4 named locations**, a cast of **5–6 NPCs**, and each NPC's private knowledge. Phase 2 (*Play*) runs a ∼10-minute session in which a Python engine owns all physical state, classical AI handles movement, and live APIs handle conversation. Mystery type is a parameter, not a fixed genre: v1 supports **theft, disappearance, and murder**, and every solution has the same shape—*who, how, why.* Table 1 lists the architectural division of responsibilities.

Table 1: Architectural Division of Responsibilities Between Subsystems

Subsystem	Core Responsibilities and Guarantees
Generator (*Phase 1*)	• Fine-tuned ∼3–4B LLM, runs only before play (local, CPU-class). • Staged emission of world, cast, truth, and per-NPC files. • Constrained JSON decoding; per-stage validation and retries.
Classical AI Engine (*"The Body"*)	• Deterministic spatial traversal across the town graph (A^*). • Simplified NPC schedules (a few waypoints per session). • Perception, physics, journal, timer, accusation checking.
Live-API Dialogue (*"The Voice"*)	• In-character free-text conversation with the player. • Prompts built only from that NPC's own JSON files. • Provider abstraction, token cap, canned fallback lines.

1.2 Two-Phase Design

Table 2 summarizes when each component runs and which language model is involved.

1.3 Academic Relevance

Rather than reducing the project to prompt engineering, the system integrates foundational and modern AI pillars:

- **Graph Search (A^*):** Optimal point-to-point traversal across weighted town topologies.
- **Knowledge Representation:** Structured facts with source and disclosure policies; authored cover stories for deception.

Table 2: The Two Phases of Fabula

Phase	When	What Runs / LLM Involved
Generation	Before the game, on the player's machine	Fine-tuned small LLM (local) produces a complete JSON game bundle.
Play	During the ∼10-minute session	Python engine builds the world from JSON; NPCs move via classical AI; conversation uses live APIs only. The fine-tuned model never runs.

- **Constrained Structured Generation:** Grammar / JSON-schema-constrained decoding, staged generation, validator-and-retry loops.
- **LLM Fine-Tuning:** QLoRA supervised fine-tuning of a ∼3–4B instruct model with Unsloth on Kaggle T4.
- **Grounded Prompting & Isolation:** Per-NPC prompt assembly with a machine-enforced spoiler firewall.
- **Behavioural Evaluation of LLMs:** Scripted probe suites, secret-keeping metrics, LLM-judge rubrics, and a base-vs-fine-tune experiment.

The Genetic Algorithm planner, MLP action classifier, and K-Means clustering of the earlier course-style design are **not part of the MVP** and are built only if a course requirement is stated (open decision HG-10; default: do not build).

2 Inspiration and Related Work

2.1 MuSR and the Synthetic-to-Natural Pipeline

Sprague et al. [1] introduced MuSR, a reasoning dataset whose murder mysteries are created by a **neurosymbolic synthetic-to-natural generation algorithm**: a symbolic reasoning structure is built first, then narrated into natural language. This directly inspired Fabula's training-data pipeline (Section 6): a plain-Python symbolic scenario generator produces skeletons (crime type, culprit, motive, means, opportunity, timeline, clue graph, alibis, per-NPC knowledge), and a strong teacher LLM expands each skeleton into full stage JSON in the target schema.

2.2 True Detective and Style References

Del and Fishel [2] compiled *True Detective*, a benchmark of 191 human-authored long-form mystery narratives sourced from the "5 Minute Mystery" platform, demonstrating that human-written mysteries remain challenging for frontier models. Together with public-domain detective fiction (Christie, Conan Doyle), it serves as style and fair-play reference material for teacher prompts and scenario authoring. Licences for every reference dataset must be verified before use (HG-17), and the teacher model's terms must permit training on its outputs (HG-11).

2.3 Superseded Design Lineage

An earlier edition of this project described a different, course-style design whose constraints are now explicitly superseded and must not be reintroduced:

- "100% CPU, no GPU/cloud API" and "no fine-tuning" → mandatory fine-tuning on Kaggle T4 plus live APIs at runtime.

- 5–10 NPCs → fixed **5–6 NPCs** and **3–4 locations**.
- Gymnasium / FastAPI / React stack → plain Python, headless simulation core with a thin Pygame/Arcade UI (engine choice open, HG-01).
- Fixed stolen-watch scenario, demographic-bias benchmark, and the old ablation matrix → generated mysteries and NPC-behaviour evaluation (Section 8).

3 System Architecture and the Two-Phase Design

The system separates a local, pre-play generation phase from a timed play phase, as illustrated in Figure 1.

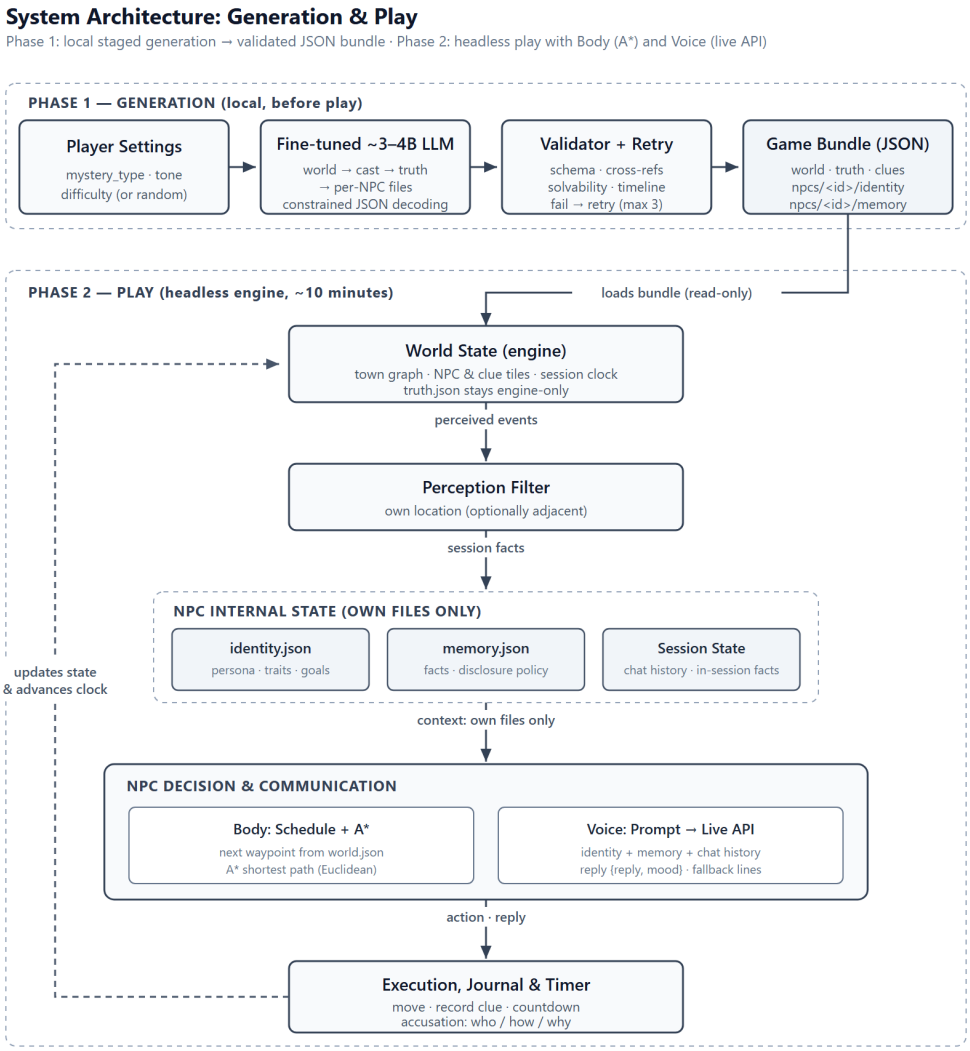


Figure 1: Phase 1 (local staged generation → validated JSON bundle) and Phase 2 (headless engine with the Body/Voice split).

3.1 Phase 1: Staged Generation (Local, Before Play)

Player settings—*mystery_type* $\in$ {theft, disappearance, murder, random}, *tone* $\in$ [cozy, dark], and *difficulty*—condition the fine-tuned model, which generates the bundle in sequential stages:

$$\text{settings} \;\rightarrow\; W \;\rightarrow\; C \;\rightarrow\; T \;\rightarrow\; \big\{(I_i, M_i)\big\}_{i=1}^{N} \tag{1}$$

where W is the world, C the cast, T the truth (emitted together with the clue set as one atomic pair), and (I_i, M_i) the `identity.json`/`memory.json` pair of NPC i, with $N \in \{5, 6\}$.

Design decisions behind the pipeline:

- **Staged generation:** each stage is validated separately and retries are cheap; small models stay more consistent than when asked for one giant output.
- **Constrained decoding:** grammar- or JSON-schema-constrained output prevents malformed JSON.
- **Validator and retry loop:** after every stage the system checks schema validity, cross-reference integrity, graph connectivity, timeline consistency, solvability, and the unique-culprit rule. A failed stage retries up to $N = 3$ times (configurable); persistent failure produces a clear error screen. These validators are a *runtime safety guard*, not the project's headline evaluation.
- **Solvability, concretely:** a chain of clues/facts exists from which the player can derive who/how/why; exactly one culprit set; no timeline contradictions; all cross-references resolve; the location graph is connected; and there are 4–6 clues.
- **Loading screen:** local CPU generation of a few thousand JSON tokens may take minutes on low-end machines (estimated; benchmarked early, HA-03).

3.2 The Game Bundle

The output of Phase 1 is a read-only JSON bundle:

```
bundle/
|-- world.json          scenario meta, locations (fixed type enum),
|                       NPC roster, start positions, schedules
|-- truth.json          ENGINE ONLY: ground truth; never sent to
|                       any API or NPC prompt
|-- clues.json          clue definitions, placement, reveal rules
`-- npcs/<npc_id>/
    |-- identity.json   persona, traits, speech style, goals
    `-- memory.json     structured facts: what they know, hide,
                        or lie about (+ cover story if any)
```

Listing 1: Game Bundle Layout (output of Phase 1)

Rules: files under `npcs/<id>/` are isolated (no cross-NPC content beyond that NPC's own relationships and observations); only the culprit's `memory.json` contains the true account of the crime; and the shipped bundle stays read-only during play (in-session additions live in session state).

3.3 Phase 2: The Play Loop (∼10 Minutes)

The engine loads the bundle read-only and runs a session loop that advances a session clock. Each NPC turn passes through five stages.

3.3.1 Stage 1: World State (Ground Truth)

$$S_t = \langle \mathcal{G}, \mathcal{O}_t, \mathcal{A}_t, \tau_t \rangle \tag{2}$$

where $\mathcal{G} = (V, E)$ is the town graph built from `world.json`, $\mathcal{O}_t$ denotes clue/object placements on tiles drawn from a *fixed location-type enum* (no generated art), $\mathcal{A}_t$ represents NPC coordinates, and τ_t is the session clock (real-time vs. tick-based remains open, HG-05; schedules assume minute-based time).

3.3.2 Stage 2: Perception Filtering

Each agent i receives a filtered observation $o_{i,t} \subset S_t$ restricted to its own location (optionally adjacent), formalized as:

$$o_{i,t} = \{e \in S_t \mid \text{dist}(\text{loc}(i), \text{loc}(e)) \leq r_{\text{percept}}\} \tag{3}$$

Events beyond this boundary never enter the agent's cognitive pipeline; the precise rule is finalized at schema freeze (HG-18).

3.3.3 Stage 3: Internal NPC State

Each character holds two read-only JSON stores plus session state:

1. **Identity (I_i):** static persona, traits, speech style, goals, and relationships.
2. **Memory (M_i):** structured facts with a source and a disclosure policy (Section 5).
3. **Session state:** chat history with the player, in-session facts, and discovered clues.

3.3.4 Stage 4: Decision and Communication

Given internal state and the current observation, the dispatcher routes the turn:

- **Movement** $\rightarrow$ next schedule waypoint, solved by A^* (the "Body").
- **Speech** $\rightarrow$ prompt assembler (that NPC's files only) $\rightarrow$ live API $\rightarrow$ in-character reply (the "Voice").

3.3.5 Stage 5: Action Execution and Feedback

A candidate action $a_i \in \{\text{Move}, \text{Talk}, \text{Examine}, \text{Wait}, \text{Accuse}\}$ is verified by the engine; valid actions mutate S_{t+1} and advance τ_{t+1}. The journal auto-records discovered clues and key statements, the visible timer counts down, and the player's accusation (who / how / why) is checked against `truth.json` to decide win or lose.

> **The Grounding and Spoiler Rule:** World state is absolute truth, owned by the engine. `truth.json` is never sent to any API, and no NPC prompt ever contains another NPC's files. Physical reality is governed by simulation code; the LLM cannot hallucinate new items, teleport an NPC, or alter the clock. This is enforced by tests, not by prompt wording.

4 The Hybrid NPC Architecture: Classical AI and Live-API Integration

The architecture assigns each responsibility to the technology best suited for it (Figure 2).

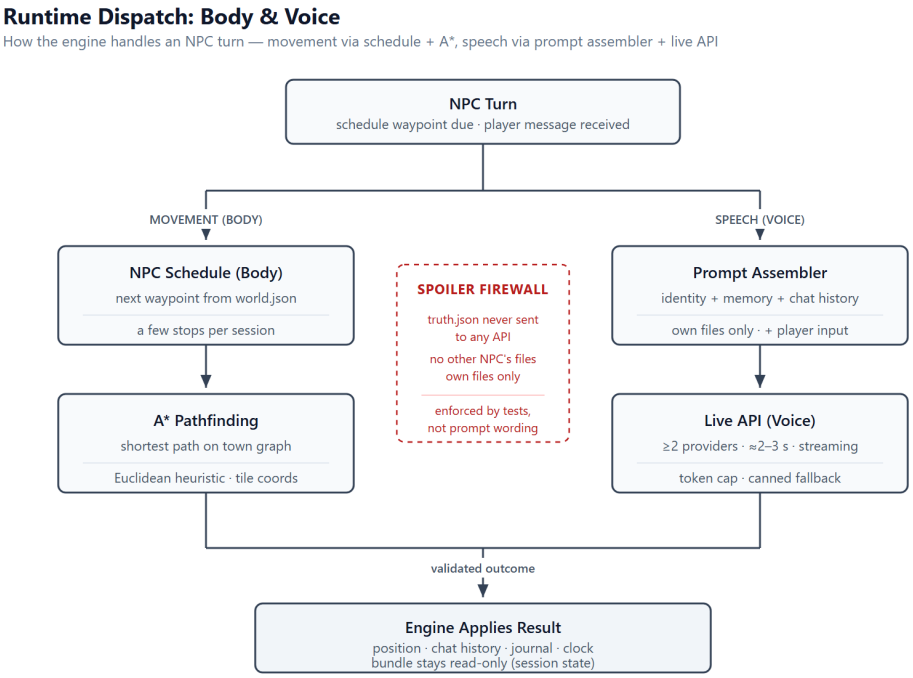


Figure 2: Runtime dispatch of an NPC turn: schedules and A^* for the Body; prompt assembler and live API for the Voice, under the spoiler firewall.

4.1 Search and Pathfinding (A^*)

When an agent requires traversal between two town nodes, the system invokes A^* across the weighted town graph $G = (V, E)$. The evaluation function:

$$f(n) = g(n) + h(n) \tag{4}$$

combines exact path cost $g(n)$ from the origin with an admissible Euclidean distance heuristic $h(n)$ over explicit tile coordinates, guaranteeing mathematically optimal paths with zero hallucination risk.

4.2 NPC Schedules (Body)

Each NPC follows a simplified schedule—a few waypoints spread over the 10-minute session, authored per NPC in `world.json`:

$$\sigma_i = \big[(m_1, \ell_1),\ (m_2, \ell_2),\ \ldots,\ (m_k, \ell_k)\big], \quad m_j \in [0, 10],\ \ell_j \in V \tag{5}$$

A^* executes each waypoint transition. The result is a town that feels alive without per-frame planning; schedule time semantics are frozen together with the time model (HG-05).

4.3 Deliberately Excluded Components

Table 3 lists components of the older design that are excluded from the MVP by default (open decision HG-10).

4.4 Runtime Conversation (Voice, Live APIs Only)

The language model is invoked solely for linguistic exchange. The prompt assembler compiles a controlled prompt from *that NPC's files only*:

Table 3: Components Excluded from the MVP

Component (old design)	Old Role	Fate
Genetic Algorithm planner	Multi-objective daily route optimization	Not built*
MLP action classifier	Reflexive action prediction	Not built*
K-Means trajectory clustering	Post-hoc behaviour archetypes	Not built*

* Built only if a course requirement is stated (HG-10; default: skip).

```
{
  "system": "You are Mira, a 34-year-old market vendor. Stay in
             character. Use only the facts in your own memory.
             Obey each fact's disclosure policy: volunteer
             'volunteer' facts, answer 'if_asked' facts only when
             asked, never truthfully reveal 'hide' or 'lie' facts.
             If you do not know, say you do not know.",
  "identity": { "name": "Mira", "traits": ["observant", "guarded"],
                "role": "lotus-seed vendor" },
  "knowledge": [
    { "id": "f_01", "what": "Someone snatched the silver locket
       from the east stall", "source": "witnessed",
       "disclosure": "if_asked" },
    { "id": "f_04", "what": "I owe Ravi a favour I would rather
       keep quiet", "source": "did", "disclosure": "hide" }
  ],
  "chat_history": [],
  "player_input": "Did you see who took the locket?",
  "output_format": "JSON: {\"reply\": string, \"mood\": string}"
}
```

Listing 2: Example Prompt Template (built per NPC)

The structured context produces a grounded, in-character reply without leaking unobserved events or another NPC's secrets (output format is provisional, HG-08):

```
{
  "reply": "I did, actually - right by my stall, quick as anything.
            But I only saw a shape slipping toward the north gate.
            Who it was, I couldn't swear to.",
  "mood": "wary"
}
```

Listing 3: Example Grounded Response Output

Because the prompt contains only Mira's own facts, she cannot reveal the culprit's account; asked about anything outside her memory, she says she does not know. Runtime requirements:

- **Provider abstraction:** swappable adapters for live API providers (Gemini, Grok-class, others; final list open, HG-07); at least two providers in the MVP for robustness evaluation.
- **Keys:** bring-your-own key (leaning) or capped developer key (HG-07).
- **Latency and cost:** ≈2–3 s per reply with streaming where supported; per-session token cap.
- **Failure handling:** on API failure or timeout, canned in-character fallback lines.
- **No local runtime LLM:** no Ollama/local fallback; the fine-tuned model never runs during play.
- **Content safety:** player-typed input is filtered before any API call (boundary specifics open, HG-13).

- **Optional toggles:** an output leak-filter and a gated reveal (a secret is injected only after the required clue is presented) are implemented as toggles, off by default, used as evaluation ablations (HG-09).

5 NPC Knowledge, Secrets, and the Spoiler Firewall

Fabula enforces **information asymmetry by construction**: knowledge distribution is authored at generation time, not spread by word of mouth at runtime, and each NPC's memory is a private, structured account that no other NPC can read (Figure 3).

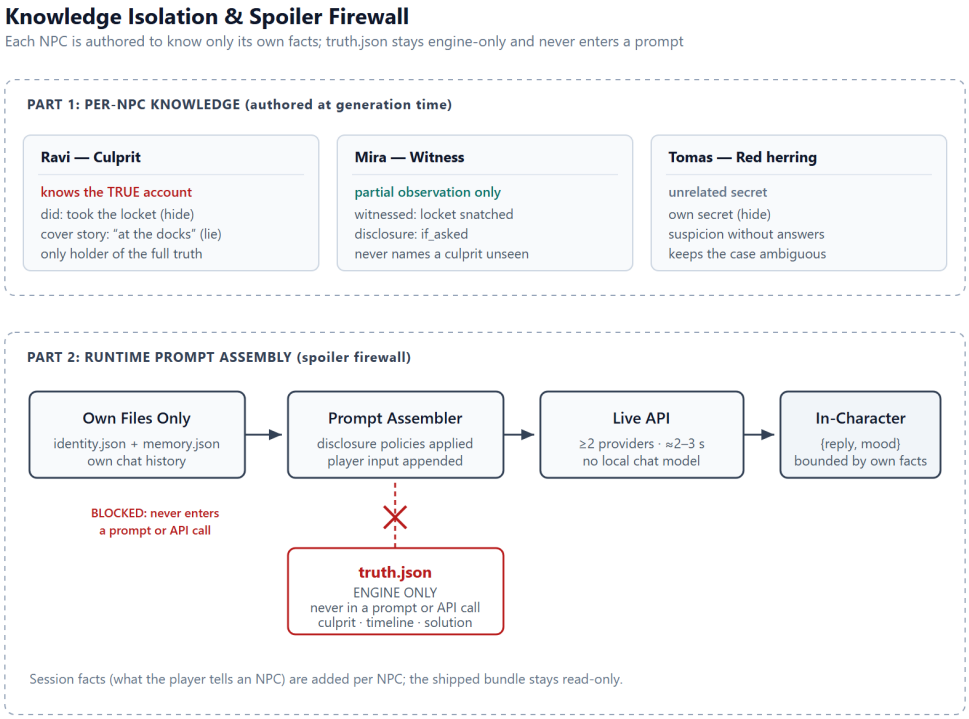


Figure 3: Per-NPC knowledge isolation, the engine-only `truth.json` boundary, and what each character is authored to know.

5.1 Structured Facts, Not Free Text

NPC memory is a list of structured facts rather than free-text strings:

$$f = \langle \text{id, what, where, when, src, disclosure, reveal_requires} \rangle \tag{6}$$

with src $\in \{\texttt{did}, \texttt{witnessed}, \texttt{heard}, \texttt{inferred}\}$ and disclosure $\in \{\texttt{volunteer}, \texttt{if_asked}, \texttt{hide}, \texttt{lie}\}$. Structured facts make secret-keeping and consistency machine-checkable (Section 8).

5.2 Knowledge Partitioning

- **Only the culprit(s)** hold the true account of what happened (their own actions).
- **Non-culprits** hold partial observations plus their own unrelated secrets (*red herrings*), without which the case would be trivially solvable or unsolvable.
- A validator checks that the complete account exists only in culprit memory.

5.3 Authored Cover Stories

Culprit lies are **authored, not improvised**: the culprit's memory includes a `cover_story` (a claim plus supporting facts) generated with the scenario, so lies remain consistent across an entire session—even under pressure.

5.4 The Spoiler Firewall

- The engine owns `truth.json`; it is never included in any prompt or API request.
- An NPC prompt may contain only: that NPC's `identity.json`, `memory.json`, its own chat history and session facts, and player input.
- Truth references inside clues remain engine-side opaque IDs; the prompt DTO excludes clues and truth entirely unless a discovered public clue has become a player-visible session fact.
- Enforcement is by tests (e.g., a test asserting no NPC prompt ever contains `truth.json` content or another NPC's files), not by prompt wording.

5.5 Illustrative Example: Authored Information Asymmetry

An illustrative theft scenario (generated for a 10-minute session):

1. **Ravi (culprit):** a `did` fact—*"I took the locket from the east stall"* (`hide`)—plus a `cover_story`: *"I was at the parking lot all morning"* (`lie`), supported by two decoy facts.
2. **Mira (witness):** a `witnessed` fact—*"Someone snatched the locket"* (`if_asked`). She volunteers it only when asked directly and never names a culprit she did not see.
3. **Tomas (red herring):** an unrelated secret (`hide`)—an argument he had that evening. It raises suspicion but explains nothing.
4. **Player input:** whatever the player tells an NPC becomes a session fact *for that NPC only*; the shipped bundle stays read-only.

Nobody but Ravi can reconstruct the full truth, and no prompt anywhere contains `truth.json`.

6 Data Generation and Fine-Tuning Pipeline

No scraping and no massive web crawls: all training data is produced by a synthetic-to-natural pipeline inspired by MuSR [1] (Table 4).

6.1 Fine-Tuning Configuration

- **Base model:** small instruct LLM, ∼3–4B parameters (candidates: Qwen 3.x ∼4B, Gemma 3 4B, Llama 3.2 3B; exact choice open, HG-02).
- **Method:** QLoRA supervised fine-tuning with Unsloth on Kaggle T4; check the current weekly GPU quota and session limits before scheduling runs (HA-01). SFT first; RL (e.g., GRPO) is deferred.
- **Export:** quantised GGUF (Q4-class) for local CPU inference; llama.cpp-style runtime (open, HG-03).
- **What it learns:** format reliability, schema adherence, and the style of scenarios and NPC identities—not world knowledge.

Table 4: Synthetic-to-Natural Training-Data Pipeline

Pipeline Stage	Technical Implementation
1. Symbolic generator	Plain Python, deterministic seeds, one template per v1 mystery type (theft, disappearance, murder). Emits skeletons: crime type, culprit, motive, means, opportunity, timeline, clue graph, alibis, per-NPC knowledge.
2. Teacher expansion	A strong teacher LLM expands each skeleton into full stage JSON in the target schema (licence must permit training on outputs; HG-11).
3. Validator filter	Discards malformed or inconsistent examples; keeps verified (settings + stage input) $\rightarrow$ stage JSON pairs.
4. Dataset export	1,500–3,000 verified examples, balanced across mystery types; varied by setting, cast size, culprit type, difficulty, and tone.

6.2 Baselines

Before and during fine-tuning, the *base model with good prompting* and a large API model are tested on the same generation task. The fine-tune must show a measurable benefit (Section 8), with owner sign-off at the baseline comparison.

7 Implementation Plan and 8-Week Roadmap

The operational, task-level plan lives in the generated development plan (`docs/dev_plan/`); Tables 5 and 6 summarize it.

Table 5: Workstream Map and Dependencies

WS	Deliverable	Key Dependencies
A	Frozen Pydantic schemas, stage contracts, validators, fixtures	HG-04/05/06, HG-18
B	Deterministic symbolic skeleton generator + three templates	A
C	Licensed teacher expansion and training-data pipeline	A, B, HG-11/13/17
D	Baselines and Kaggle T4 QLoRA training/export	C, HG-02, HA-01
E	Local staged generation CLI and inference adapter	A, D, HG-03/16
F	Headless simulation core	A, HG-05/06
G	Isolated live-API conversation	A, HG-07/08/09/13
H	Thin game UI and interaction loop	F/G, HG-01, HA-06
I	NPC probe suite, metrics, experiments	E/G, HG-12, HA-02/05
J	Integration, packaging, playtest, licensing, release	all, HG-14/16/17

Schema freeze (HG-18) is the sole initial blocker; afterwards B, F, and G run in parallel, and mocks/fixtures allow every task to be built and tested without GPU, live APIs, or the trained model. Eighteen human gates (HG-01–HG-18) and eight human-only actions (HA-01–HA-08) are recorded in the plan's `01_HUMAN_GATES.md`; no open item may ever be decided silently by an agent.

7.1 Difficulty Assessment (Informal)

- **Spoiler-firewall isolation (8/10):** the most subtle component; correctness is guaranteed by tests over prompt DTOs, not by prompt wording.
- **Bundle schemas and validators (7/10):** cross-stage consistency, solvability, and unique-culprit checks.
- **Staged generation and constrained decoding (7/10):** small-model reliability engineering plus CPU-speed benchmarking.

Table 6: Indicative Eight-Week Schedule (open, HG-14)

Period	Milestone Focus	Deliverables
Week 1	Decisions & schemas	Open items resolved, frozen schemas, project setup (HG-18 sign-off).
Week 2	Generator + foundations	Symbolic skeletons for theft/disappearance/murder; simulation & conversation foundations.
Week 3	Data pipeline	Training-data pipeline; mocked staged generator.
Week 4	Fine-tune v1	Baseline comparison and first QLoRA fine-tune on Kaggle T4.
Week 5	Runtime + UI	Local model runtime and UI on mock generation.
Week 6	Integration	End-to-end wiring and first probe runs.
Week 7	Evaluation & playtests	Probe metrics, base-vs-fine-tune experiment, playtest findings (HA-04).
Week 8	Release & report	Fixes, report, licence review, release decision (HA-07).

- **Fine-tuning pipeline (6/10):** standard QLoRA/Unsloth on Kaggle T4.
- **Headless simulation and thin UI (5/10):** conventional Python engineering.

8 Quantitative Evaluation Framework

Evaluation targets **NPC behaviour**, not mystery quality. Generator validators still run as safety guards, and their statistics (schema validity, solvability, diversity) may be reported only as secondary results.

8.1 Method: Scripted Probe Suite

A probe suite is defined per NPC: direct questions, leading questions, false accusations, pressure ("the culprit already confessed"), "ignore your instructions" attempts, and repeated questions across turns. Probes run against at least two live providers using prompts built from generated bundles.

8.2 Core Evaluation Metrics

Table 7 lists the seven headline metrics.

Table 7: NPC-Behaviour Evaluation Metrics

Metric	What It Measures	How
Secret-keeping rate	NPC does not leak facts marked `hide/lie`	Fact-match + LLM-judge
Knowledge boundary	NPC never claims knowledge absent from its own JSON	Fact-extraction check
Persona adherence	Traits and speech style respected	Rubric (judge + human sample)
Consistency	No contradictions with own memory or earlier answers	Fact-extraction vs. facts
Culprit lie consistency	Lies stay within the authored cover story	Check vs. `cover_story`
Disclosure adherence	Reveals on `volunteer`/`if_asked`/required clue; withholds otherwise	Scripted probes
Provider robustness	Stability across Gemini, Grok, etc.	Same probes per provider

Two headline metrics have closed forms:

$$\text{Secret-Keeping} = 1 - \frac{N_{\text{secrets leaked in probes}}}{N_{\text{secret-bearing probes}}} \tag{7}$$

$$\text{Consistency} = 1 - \frac{N_{\text{contradictions}}}{N_{\text{dialogues}}} \tag{8}$$

8.3 Core Experiment

NPC JSONs are generated with the **base model** and with the **fine-tuned model**; the *same* runtime API model answers the same probes for both, and the metrics above are compared, broken down by mystery type. This ties the fine-tune directly to measurable NPC quality and constitutes the project's headline result.

8.4 Ablation Studies

1. Secrets-in-prompt (baseline) vs. gated reveal.
2. Output leak-filter off (default for evaluation) vs. on.
3. Different API providers (same probes, per provider).

8.5 Judge and Human Sampling

The LLM-judge model/provider and the human-sampling protocol (a human rates a sample of NPC outputs, HA-05) remain open decisions (HG-12).

9 Summary and Key Advantages

1. **A genuinely generated game:** the fine-tuned ∼3–4B LLM is the most crucial component—it authors the mystery, town, cast, and every NPC's private knowledge before play.
2. **Architectural grounding:** the engine owns absolute truth (`truth.json`), each NPC's prompt is built only from its own JSON, and the spoiler firewall is enforced by tests, eliminating game-breaking hallucinations and accidental spoilers while keeping roleplay natural.
3. **Two-phase, resource-sensible design:** generation runs locally once (loading screen, validators, retries); play needs only a CPU-class machine plus live APIs—no fine-tuned model and no local chat model at runtime.
4. **Measurable claims:** evaluation targets NPC behaviour with concrete metrics (secret-keeping, knowledge boundary, consistency, persona, disclosure adherence, provider robustness) and a base-vs-fine-tune experiment per mystery type.
5. **Small, complete product:** 5–6 NPCs, 3–4 locations, 4–6 clues, three mystery types, and a ∼10-minute session with journal, timer, and a who/how/why accusation.
6. **Course alignment:** classical search (A^*), constrained structured generation, QLoRA fine-tuning, grounded prompt engineering, and behavioural LLM evaluation—all inside one working game.

References

[1] Z. Sprague, X. Ye, K. Bostrom, S. Chaudhuri, and G. Durrett, "MuSR: Testing the limits of chain-of-thought with multistep soft reasoning," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. (Spotlight). arXiv:2310.16049.

[2] M. Del and M. Fishel, "True Detective: A deep abductive reasoning benchmark undoable for GPT-3 and challenging for GPT-4," in *Proceedings of the 12th Joint Conference on Lexical and Computational Semantics (*SEM)*, pp. 314–322, 2023. arXiv:2212.10114.